OntoDB

Database Internals by Example

Storage, indexing, query processing, concurrency, and recovery,
taught through a database management system for ontologies

Lam Nguyen

Contents

Preface **v**

I Foundations **1**

1 What a Database Engine Is **2**

1.1 The layers . . . 2
1.2 RDF in one page . . . 3
1.3 SPARQL in one page . . . 3
1.4 The running example . . . 4

II Storage **5**

2 Disks, Files, and Pages **6**

2.1 The storage hierarchy . . . 6
2.2 Pages . . . 6
2.3 Slotted pages . . . 7
2.4 Talking to the file system . . . 8

3 The Buffer Pool **9**

3.1 Frames, the page table, and pins . . . 9
3.2 Choosing a victim . . . 10
3.3 Steal and force . . . 10

4 B-Trees and B+ Trees **12**

4.1 Why a tree of pages . . . 12
4.2 B-tree versus B+ tree . . . 12
4.3 Search . . . 13
4.4 Insertion and splits . . . 13
4.5 Deletion . . . 13
4.6 Bulk loading . . . 14
4.7 Iterators and pins . . . 14

5 Indexing RDF **16**
5.1 Dictionary encoding . . . 16
5.2 A key whose bytes sort correctly . . . 17
5.3 Prefix scans . . . 17
5.4 One ordering is not enough . . . 17

6 The Log-Structured Alternative **19**
6.1 How an LSM tree works . . . 19
6.2 Three amplifications . . . 19

III Query Processing **21**

7 From SPARQL to a Plan **22**
7.1 Parsing and binding . . . 22
7.2 Logical and physical plans . . . 22
7.3 The iterator (Volcano) model . . . 23
7.4 The operators . . . 23

8 Joins and Query Optimization **25**
8.1 Three join algorithms . . . 25
8.2 Cardinality estimation . . . 25
8.3 Searching for a join order . . . 26
8.4 Explain analyze . . . 26

IV Transactions **28**

9 Transactions and Isolation **29**
9.1 ACID . . . 29
9.2 Schedules and serializability . . . 29
9.3 Anomalies . . . 29
9.4 Isolation levels . . . 30

10 Concurrency Control **32**
10.1 Two-phase locking . . . 32
10.2 Locking at several granularities . . . 32
10.3 Deadlock . . . 33

10.4 Phantoms . . . 33
10.5 Latches are not locks . . . 33
10.6 The alternative: multiversion concurrency . . . 34

V Recovery **35**

11 Logging and the Write-Ahead Rule **36**
11.1 The failure model . . . 36
11.2 Undo and redo . . . 36
11.3 The write-ahead rule . . . 37
11.4 Making commits cheap: group commit . . . 37
11.5 When a record is damaged . . . 37

12 ARIES **39**
12.1 The data structures . . . 39
12.2 Checkpoints . . . 39
12.3 Pass 1: Analysis . . . 39
12.4 Pass 2: Redo . . . 40
12.5 Pass 3: Undo . . . 40
12.6 A worked example . . . 40
12.7 Structure changes: nested top actions . . . 41

13 Durability in Practice **43**
13.1 Where it sits in the steal/force table . . . 43
13.2 Logical logging . . . 43
13.3 One statement, one transaction . . . 44
13.4 Recovery . . . 44
13.5 Checkpoints as atomic snapshots . . . 44
13.6 Damaged logs: recovery modes . . . 45
13.7 Verification . . . 45
13.8 Testing a recovery design . . . 45

A A Map from Chapters to OntoDB **47**

Bibliography **48**

Preface

A database engine is a small number of ideas, applied with care. Data lives in fixed-size pages because disks move blocks. A buffer pool keeps the hot pages in memory because memory is fast and small. A B+ tree turns a key into a page in three or four reads because its fanout is in the hundreds. A lock manager makes concurrent transactions look sequential. A write-ahead log, and an algorithm called ARIES, make a committed transaction survive a crash that happens at any instant.

This book explains those ideas from the ground up, and it explains each one twice: first as theory, then as it appears in a real system. The real system is **OntoDB**, a database management system for ontologies. Its data is RDF, a graph of subject–predicate–object triples, and its query language is SPARQL. Every chapter ends its theory with an *In OntoDB* box that shows the decision OntoDB made, and why.

Why an ontology database

A relational engine teaches the same ideas, so why a triple store? Two reasons. First, a triple store strips the problem to its core: there is one table, with three columns, and every interesting question is a join. That makes indexing and join ordering impossible to wave away. Second, ontologies are where correctness matters most and is checked least: a scientific ontology edited by many hands needs transactions, a log, and recovery just as much as a bank ledger does.

How to read this book

The parts follow the path of a request through the engine, from the bottom up.

Foundations What an engine is made of, and enough RDF and SPARQL to follow the examples.

Storage Pages, the buffer pool, B+ trees, how RDF is indexed, and the log-structured alternative.

Query processing How a SPARQL query becomes a tree of operators, and how joins are chosen.

Transactions What isolation means, and how locking provides it.

Recovery Logging, the write-ahead rule, ARIES, and how OntoDB makes its databases durable today.

Code appears only where a picture or a sentence would be less precise: a key layout, a log record, a few lines of pseudocode for an algorithm. The aim is to understand *why*, not to read an implementation.

Status

OntoDB is built feature by feature, and parts of this book describe designs that are specified but not yet implemented. Each chapter's *Status* box says which. At the time of writing, the SPARQL shell, the log records, the log manager, and durable databases (snapshots, a checksummed write-ahead log, recovery, and verification) run; the buffer pool, the B+ tree, the indexed store, the optimizer, concurrency control, and page-level ARIES are designed and being built.

Acknowledgments

The structure of this book owes much to Alex Petrov's *Database Internals*, whose division into storage engines and distributed systems it follows for the first half. The recovery chapters follow Mohan et al.'s ARIES paper closely, because nothing explains ARIES better than ARIES.

Part I

Foundations

Chapter 1

What a Database Engine Is

A database management system (DBMS) accepts declarative requests, such as "find every student and their name", and answers them correctly and quickly, while many clients read and write at once and while machines fail. Each of those words is a subsystem. *Correctly* under concurrency is the job of the transaction manager. *Quickly* is the job of indexes and the query optimizer. *While machines fail* is the job of the log and recovery.

1.1 The layers

Almost every engine, relational or not, is built from the same layers. A request travels down them, and data travels up.

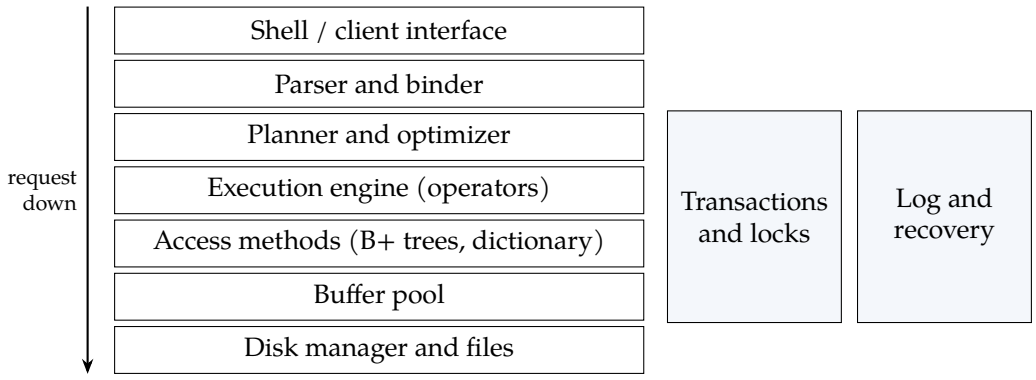


Figure 1.1: The layers of a database engine. Transactions and recovery cut across the stack.

Interface Reads text, returns results. For OntoDB this is the `ontodb` shell.

Parser and binder Turn text into a tree, check names, and replace each constant with an internal id.

Planner and optimizer Choose *how* to answer: which index to use, in what order to join, with which algorithm.

Execution engine Runs the chosen plan as a tree of operators that pass rows to each other.

Access methods The data structures that find data: B+ trees for ranges, hash tables for exact matches, the dictionary for term ids.

Buffer pool Caches disk pages in memory, decides which to evict, and enforces the write-ahead rule when a page goes back to disk.

Disk manager Reads and writes fixed-size pages of a file.

Two subsystems do not fit in one layer. The **transaction manager** and **lock manager** touch every read and write. The **log manager** and **recovery** touch every update, every page write, and every restart. Most of the difficulty of building an engine lives in how these cross-cutting pieces interact with the layers.

Key idea

Each layer hides a cost from the one above. The buffer pool hides the disk; the B+ tree hides how many pages a lookup needs; the optimizer hides that a query can be answered in a thousand ways. Understanding an engine means knowing what each layer hides, and when the hiding fails.

1.2 RDF in one page

RDF represents knowledge as a set of *triples*. A triple is (*subject*, *predicate*, *object*): the subject is a thing, the predicate a relationship, and the object another thing or a value.

```
<http://ex.org/Alice>  <http://ex.org/name>       "Alice"
<http://ex.org/Alice>  rdf:type                   <http://ex.org/Student>
<http://ex.org/Metal>  rdfs:subClassOf            <http://ex.org/Material>
```

Things are named by IRIs (`<http://...>`); values are literals, optionally with a language (`"Métal"@fr`) or a datatype (`"21"^^xsd:integer`). An *ontology* is RDF that describes concepts and their relations: `Metal subClassOf Material`, `Alloy subClassOf Metal`. OWL, the standard ontology language, is itself written as RDF triples, so a triple store can store any OWL ontology without understanding it.

1.3 SPARQL in one page

SPARQL asks questions by *pattern*. A *triple pattern* is a triple in which some positions are variables. A *basic graph pattern* is a set of triple patterns that share variables; its answers are every way of binding the variables so that every pattern becomes a stored triple.

```
PREFIX ub: <http://example.edu/univ#>
SELECT ?name WHERE {
  ?s a ub:Student .        # ?s is a student
  ?s ub:name ?name .       # ... and has a name
}
```

Two patterns that share `?s` are a *join*. That is the single most important fact about evaluating SPARQL: a query with k patterns is a k-way self-join of one large table of triples. Everything in chapter 5 and chapter 8 follows from it.

1.4 The running example

In OntoDB

OntoDB is a database management system for ontologies, written in C++20. It loads Turtle, N-Triples, and RDF/XML (so OWL files load as triples), and it answers SPARQL basic graph patterns with filters, `INSERT DATA`, and `DELETE DATA`. Its layers map one to one onto fig. 1.1: a shell (`tools/shell`), a lexer, parser, and binder (`src/parser`, `src/binder`), a planner and optimizer, volcano executors (`src/execution`), a dictionary and a store (`src/dictionary`, `src/store`), a buffer pool and disk manager (`src/buffer`, `src/storage`), and a log and recovery manager (`src/recovery`).

A single query, followed through OntoDB, touches every layer:

1. The shell reads lines until the query's braces balance.
2. The parser builds a syntax tree; the binder resolves prefixes, turns every IRI and literal into a 64-bit id from the dictionary, and gives each variable a column number.
3. The planner builds a tree: one scan per triple pattern, joins between them, a filter, a projection.
4. The execution engine pulls rows of ids up that tree, one at a time.
5. The shell turns ids back into text through the dictionary and prints them.

An insert adds a sixth step: before the shell says `inserted 1 triple`, a log record describing the insert must be on disk. Why that order is non-negotiable, and how it lets the engine survive a crash, is the subject of part V. First, storage.

Part II

Storage

Chapter 2

Disks, Files, and Pages

Every design decision in a storage engine starts from one fact: memory is fast and volatile, and storage is slow and durable. A database must keep its data on the slow, durable device and still answer queries at the speed of the fast, volatile one. This chapter is about the unit that moves between the two: the *page*.

2.1 The storage hierarchy

Device	Typical access latency	Survives power loss
CPU cache	1–10 ns	no
Main memory (DRAM)	about 100 ns	no
NVMe SSD, random 4 KiB read	10–100 μs	yes
Spinning disk, random read	5–10 ms	yes

Table 2.1: Orders of magnitude, not benchmarks. The gap between memory and storage is a factor of a hundred to a hundred thousand.

Two properties of storage shape everything above it.

Block granularity. A device reads and writes whole blocks (typically 512 bytes or 4 KiB). Reading one byte costs a whole block. So the engine should arrange data so that one block read brings many useful bytes.

Sequential beats random. Even on SSDs, sequential access is cheaper than random access, and on spinning disks it is dramatically cheaper. An engine that writes sequentially (a log) and reads mostly through an index (a tree) uses the device the way it wants to be used.

2.2 Pages

A *page* is the engine's unit of I/O: a fixed-size block of the database file, usually 4, 8, or 16 KiB, aligned to the device's block size. The engine never reads half a page and never writes half a page. Fixing the size has three payoffs:

- **Addressing is arithmetic.** Page i lives at byte offset $i \times$ PAGE_SIZE in the file. A page id is all the engine needs to find it.
- **Memory is interchangeable.** Every buffer-pool frame holds exactly one page, so any frame can hold any page (chapter 3).

- **Failure is bounded.** A write either lands a whole page, or, if power fails mid-write, produces a *torn page*: part old, part new. Because the size is fixed and known, a checksum over the page detects the tear.

Key idea

A page id is a stable name. Pointers inside the database (a B+ tree child, a heap record's location) are page ids, never memory addresses, so they remain valid across restarts and across evictions.

2.3 Slotted pages

Fixed-size records are easy to place: record k starts at byte $k \times$ size. Real data is variable-sized: an IRI might be 20 bytes or 200. The standard answer is the *slotted page*.

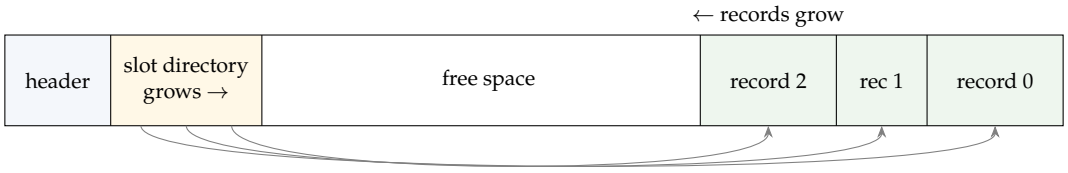


Figure 2.1: A slotted page. The slot directory grows forward from the header, records grow backward from the end, and free space sits between them.

The header records how many slots exist and where free space begins. Each slot holds a record's offset and length. A record is addressed by a *record id* (RID): (*page id*, *slot number*). Two properties make this work:

- **Records can move without changing their RID.** To compact the page, slide records together and update the slot offsets. The slot number, and therefore the RID, is unchanged.
- **Deletion is cheap.** Mark the slot free; a later insert may reuse it. The space is reclaimed when the page is compacted, or when a new record fits in the hole.

In OntoDB: pages and the term heap

Every OntoDB page is 4096 bytes (`PAGE_SIZE`). The first 8 bytes of *every* page, from the very first version, are reserved for the *pageLSN*, the log sequence number of the last update applied to that page; the payload starts at byte 8. Reserving the field before any logging existed means the file format did not have to change when the write-ahead log arrived (chapter 11).
The spellings of IRIs and literals live in a *term heap* made of slotted pages. The B+ tree leaves never contain strings: they contain 24-byte keys of three ids. That separation keeps the tree's fanout high, and it is why a long IRI costs nothing in the index (chapter 5). A spelling is never deleted when a triple using it is deleted: the dictionary keeps the id, so a log record written earlier can still name it.

2.4 Talking to the file system

An engine reads and writes its file through the operating system, and two details of that conversation decide whether data survives a crash.

The page cache. A `write` call returns as soon as the bytes are in the operating system's page cache, in memory. They reach the device later. A crash after `write` returns can still lose them.

`fsync`. Calling `fsync` on a file asks the operating system to push that file's dirty data to the device and wait until the device reports it durable. It is the only promise of durability the engine gets, and it is expensive: a few milliseconds on a disk, tens to hundreds of microseconds on an SSD. Two consequences follow.

1. **Batch the expensive call.** Engines amortize `fsync` across many transactions; the log manager's *group commit* (chapter 11) exists for this.
2. **Never assume a failed `fsync` can be retried.** On several operating systems, after `fsync` reports an error the kernel may already have discarded the dirty pages and cleared the error, so a second `fsync` that "succeeds" has lost data. PostgreSQL learned this in 2018 and now treats a failed `fsync` as fatal [23, 22].

Atomic replacement. Renaming a file over another is atomic on POSIX file systems: a reader sees the old file or the new one, never a mixture. The idiom "write to a temporary file, `fsync` it, `rename` it over the target, `fsync` the directory" replaces a whole file atomically. OntoDB writes its snapshots this way (chapter 13).

Status

The page frame and the pageLSN layout exist. The disk manager, which allocates pages and reads and writes them, and the term heap are being built (plans 1.1 and 3.1).

Further reading

Petrov [1], chapters 2 and 3, on file formats and slotted pages. Pillai et al. [22] on what file systems actually guarantee.

Chapter 3

The Buffer Pool

The database is larger than memory, but queries must run at memory speed. The *buffer pool* bridges the two: a fixed array of in-memory *frames*, each able to hold one page, plus the bookkeeping that decides which pages are in memory and when they go back to disk.

3.1 Frames, the page table, and pins

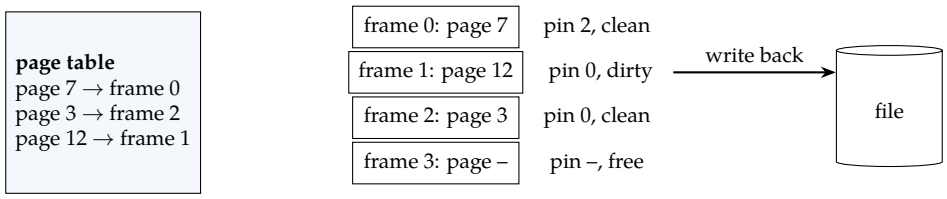


Figure 3.1: A buffer pool with four frames. The page table maps page ids to frames; each frame has a pin count and a dirty bit.

Each frame carries two pieces of state:

Pin count. How many threads are using the page right now. A pinned page must not be evicted: someone holds a pointer into it.

Dirty bit. Whether the in-memory copy differs from the one on disk. A dirty page must be written before its frame is reused.

The interface follows from that:

- `FetchPage(id)`: if the page is in a frame, increment its pin and return it (a *hit*); otherwise choose a victim frame, write it back if dirty, read the page into it, pin it (a *miss*).
- `UnpinPage(id, dirty)`: decrement the pin; mark the frame dirty if the caller changed it.
- `NewPage()`: allocate a page id on disk and return a zeroed, pinned frame.
- `FlushPage(id)`: write a dirty page now.

Key idea

A pin is a promise, and a leaked pin is a leak of memory the pool can never reclaim. Engines therefore wrap pins in a scoped object (a *page guard*) that unpins exactly once when it goes out of scope.

3.2 Choosing a victim

When every frame is occupied and a miss arrives, the pool must evict an unpinned page. The choice matters: a bad policy turns hits into misses, and every miss is a disk read.

LRU. Evict the least recently used page. Simple, and good for many workloads. Its weakness is *sequential flooding*: one large scan touches every page once, and each touch makes a cold scan page look "recent", so the scan flushes out the hot index pages that every other query needs.

CLOCK. An approximation of LRU that avoids moving pages in a list on every access: each frame has a reference bit set on access, and a "clock hand" sweeps the frames, clearing bits and evicting the first frame whose bit is already clear. Cheap, and still vulnerable to flooding.

LRU-K. O'Neil, O'Neil, and Weikum's fix for flooding [9]. Remember the last K access times of each page. The *backward K-distance* of a page is the time since its K-th most recent access; a page accessed fewer than K times has infinite distance. Evict the page with the largest distance.

With $K = 2$, a page touched once by a scan has infinite distance, while an index root touched thousands of times has a small one. The scan page goes first. LRU is the special case $K = 1$.

Page	Accesses (time)	2nd most recent	Backward 2-distance at $t = 10$
A (index root)	1, 4, 8, 9	8	2
B (index leaf)	3, 6	3	7
C (scanned once)	7	—	∞

Table 3.1: LRU-K with $K = 2$. LRU would evict B (least recently used at time 6); LRU-K evicts C, the page with no second access, even though C was touched more recently than B.

When several pages have infinite distance, a tie-breaking rule is needed; evicting the one whose most recent access is oldest is the usual choice.

3.3 Steal and force

Two policy questions connect the buffer pool to recovery, and their answers decide what the log must contain.

Steal? May the pool write a dirty page of an *uncommitted* transaction to disk, to free its frame? If yes, the disk can hold changes that must later be *undone*.

Force? Must every page a transaction changed be written to disk *before* it commits? If no, a committed change may exist only in memory at the time of a crash, and must later be *redone*.

Steal lets the pool evict any unpinned page, so a long transaction cannot exhaust memory. No force lets a commit write one small log record instead of every page it touched. Both are wins, and chapters 11 and 12 show how the log makes them safe.

	Force	No force
No steal	no undo, no redo (slow, memory-bound)	redo only
Steal	undo only	undo and redo (fast, flexible)

Table 3.2: The four combinations. High-performance engines choose steal and no force, and pay for it with a log that supports both undo and redo.

In OntoDB: the buffer pool

OntoDB's pool uses **LRU-K** with $K = 2$ by default (`\set lru_k`), tie-breaking by the older K-th access. Its policy is **steal / no force**. Pages are reached only through `ReadPageGuard` (shared latch, one pin) and `WritePageGuard` (exclusive latch, one pin); both unpin exactly once. The test harness asserts that the number of pinned frames is zero after every query and every destroyed iterator: a leaked pin fails the test even when the query's rows were right.
When the pool writes a dirty page and a log manager is installed, the order is fixed: first flush the log up to the page's pageLSN, then write the page. A named crash point sits between the two, so a test can kill the process after the log is durable and before the page is. That ordering is the write-ahead rule of chapter 11.

Status

The LRU-K replacer, the pool, and the guards are being built (plans 1.3 to 1.5). The experiment that checks them is simple: run a scan with a pool of 4 frames and of 64, with $K = 1$ and $K = 2$, and compare hits and misses in `\stats`.

Further reading

Petrov [1], chapter 5 on buffer management. O'Neil et al. [9]. Effelsberg and Härder [10] on buffer management principles.

Chapter 4

B-Trees and B+ Trees

An index answers "where is the data with this key?" without reading all the data. For data on disk the answer has been the same since 1970: the B-tree of Bayer and McCreight [5], and its variant the B+ tree, described by Comer as ubiquitous [6]. This chapter explains why it fits disks so well, and how it stays balanced.

4.1 Why a tree of pages

A binary search tree has fanout 2: each node has two children, so finding one of N keys takes about $\log_2 N$ steps. If every step is a page read, a billion keys need 30 reads. A B-tree makes each node a whole *page* and packs as many keys into it as fit. With fanout F, a lookup takes $\log_F N$ reads.

Fanout F	Height for 10^9 keys	Height for 10^{12} keys
2	30	40
100	5	6
500	4	5

Table 4.1: Height is $\lceil \log_F N \rceil$. A large fanout turns a billion-key lookup into four page reads, and the top levels are almost always in the buffer pool.

The upper levels are tiny (a tree of fanout 500 has 1 root and 500 children above a quarter-million leaves) and hot, so they stay in the buffer pool. In practice a lookup costs one or two actual disk reads.

Key idea

A B-tree is a binary search tree redesigned for block devices: make each node a page, make the fanout as large as the page allows, and the tree is shallow enough that its top fits in memory.

4.2 B-tree versus B+ tree

A *B-tree* stores keys and values in every node. A *B+ tree* stores values only in the *leaves*; internal nodes hold only *separator keys* that direct the search. Two consequences favor the B+ tree for databases:

- **Higher fanout.** Internal nodes hold no values, so more separators fit per page and the tree is shallower.

- **Fast range scans.** Leaves are linked to their right sibling. A range query finds its first key with one descent, then walks sideways along the leaf chain without climbing back up.

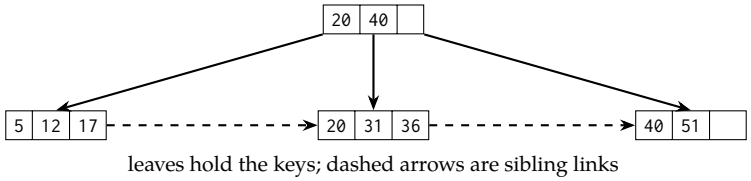


Figure 4.1: A B+ tree. Keys below separator 20 are in the left child; keys from 20 up to 40 in the middle; 40 and above in the right.

The *separator rule* is the tree's central invariant: in an internal node with separators $k_1 < k_2 < \cdots < k_n$ and children $c_0, \ldots, c_n$, every key in c_{i-1} is less than k_i, and every key in c_i is at least k_i.

4.3 Search

Start at the root. At each internal node, binary-search the separators to pick the child whose range contains the key, and descend. At the leaf, binary-search for the key. A range scan $[\ell, u)$ searches for ℓ, then follows sibling links until a key reaches u.

4.4 Insertion and splits

Insert into the leaf the search reaches. If the leaf has room, done. If it is full, *split* it:

1. Allocate a new leaf. Move the upper half of the keys into it.
2. Link it: the new leaf's sibling is the old leaf's sibling, and the old leaf's sibling becomes the new leaf.
3. Insert a separator into the parent: the first key of the new leaf, which is *copied up* (it stays in the leaf).
4. If the parent is now full, split it too. An internal split *pushes up* its middle key: the key moves to the grandparent and is removed from both halves.
5. If the root splits, create a new root with two children. This is the only way the tree grows taller, so every leaf is always at the same depth.

A classic bug lives in step 3: if the middle key is copied up *and* left in the *left* leaf, the separator rule breaks, because the right child's keys must be at least the separator while a copy of it sits on the left. Writing an invariant checker before writing insert catches this at once.

4.5 Deletion

Remove the key from its leaf. If the leaf is still at least half full, done. Otherwise:

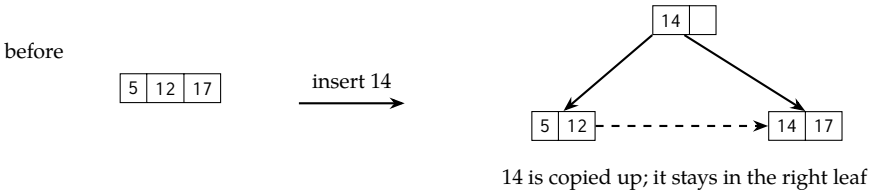


Figure 4.2: A full leaf with capacity 3 splits on insert. The new right leaf's first key becomes the separator.

- **Redistribute**: if a sibling has more than half, move a key across and update the parent's separator, which must now reflect the new boundary.
- **Merge**: otherwise combine the leaf with a sibling, remove the separator between them from the parent, and free the empty page. The parent may now underflow, and the process repeats upward; if the root is left with one child, that child becomes the root.

Many production systems relax deletion and allow underfull nodes, merging only when a node empties: deletes are rarer than inserts, and a tree that gets a little sparse rarely matters.

4.6 Bulk loading

Building a tree by inserting keys one at a time leaves leaves about two thirds full and touches pages in random order. When the keys are available up front, *bulk loading* is far better: sort the keys, fill leaves completely from left to right, link them, then build each internal level from the first keys of the level below. One sequential pass per level, and every page is full.

4.7 Iterators and pins

A range scan returns rows one at a time to the query engine, so it is an *iterator* that holds a position: the current leaf and an index inside it. The leaf must stay pinned while the iterator points into it. When the iterator moves to the sibling, it should unpin the current leaf *before* pinning the next one; holding both, plus a parent, can exhaust a small buffer pool and deadlock.

In OntoDB: the B+ tree

OntoDB's index is a B+ tree. Its key is a 24-byte `TripleKey` (chapter 5), and the leaf stores the key itself: the triple *is* the record, so there is no separate tuple id. Leaves link to their right sibling, the header page stores the root (or `INVALID_PAGE_ID` for an empty tree), and a scan holds at most one pinned leaf at a time.
An invariant checker (`CheckInvariants`) verifies sizes, key order, sibling links, and the separator rule, and the shell can draw the tree (`\tree spo dot`). Splits and merges are multi-page operations, so they have named crash points where a recovery test may kill the process; chapter 12 explains why a split is redone but never undone. Bulk load sorts ids, not strings, because the tree's order is the order of the ids.

Status

The B+ tree is the next major build (plans 2.1 to 2.7): layout, invariants, search, insert and split, iterator, bulk load, and delete.

Further reading

Bayer and McCreight [5]; Comer [6]; Petrov [1], chapters 2, 4, and 6.

Chapter 5

Indexing RDF

A relational table has a schema, and each column can be indexed on its own. An RDF store has one table with three columns, and a query can bind any subset of them. This chapter shows how a triple store turns that into fixed-size keys in B+ trees: first by replacing strings with integers, then by laying out the integers so that byte order is query order, and finally by keeping more than one ordering of the same triples.

5.1 Dictionary encoding

IRIs are long and repeated: the same predicate appears in millions of triples. Storing strings in the index would waste space and make every comparison a string comparison. Instead, a *dictionary* maps each distinct term to a dense integer id, once.

id	canonical spelling
1	`<http://example.edu/univ#Alice>`
2	`<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>`
3	`<http://example.edu/univ#Student>`
4	`"Alice"`

A triple becomes three integers, `(1, 2, 3)`. Comparing ids is one machine instruction, and the index stores 24 bytes per triple regardless of how long the IRIs are.

Two rules keep the dictionary correct:

- **One spelling per term.** The loader and the query binder must produce the same canonical text for the same term, or one IRI becomes two ids and every join on it silently fails. Escapes, datatype IRIs, and language tags are part of the spelling.
- **Ids never change.** An id assigned once keeps its meaning across restarts. A log record that names id 18 must still mean the same term when it is replayed (chapter 13).

In OntoDB: the dictionary

Ids are 64-bit and start at 1; 0 is never a valid id, so a zeroed key never names a real term, and the maximum value means "unbound". A single codec produces canonical spellings for both the loader and the binder: `<iri>`, `"lex"`, `"lex"@en`, `"lex"^^<datatype>`, and `_:b0`. Numeric literals compare by value in filters; everything else compares by id for equality and by spelling for order.

5.2 A key whose bytes sort correctly

A B+ tree compares keys. The fastest comparison is `memcmp` on raw bytes, which works only if byte order equals the intended order. For a key made of three unsigned integers, that requires **big-endian** encoding: the most significant byte first.

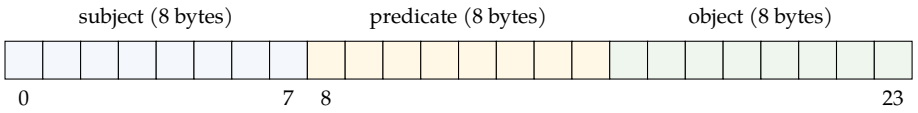


Figure 5.1: A 24-byte triple key: three big-endian unsigned 64-bit ids.

Why big-endian: little-endian stores the least significant byte first, so the id 256 (`00 01 00...` in little-endian) would compare below the id 1 (`01 00 00...`). With big-endian, integer order and byte order agree, and the comparison needs no decoding at all.

5.3 Prefix scans

A triple pattern with some positions bound is a *prefix* of the key, if the bound positions come first. "Every triple with subject 5" is every key whose first 8 bytes encode 5. Such a prefix is a contiguous range of keys, so a scan is one descent plus a walk along leaves. The range is $[\text{lower}, \text{upper})$, where lower is the prefix padded with zeros and upper is the prefix's last component plus one:

Bound components	Lower bound	Upper bound (exclusive)
0	`(0, 0, 0)`	past every key
1	`(5, 0, 0)`	`(6, 0, 0)`
2	`(5, 7, 0)`	`(5, 8, 0)`
3	`(5, 7, 9)`	`(5, 7, 10)`

5.4 One ordering is not enough

An index ordered subject, predicate, object (SPO) serves any pattern whose bound positions are a prefix of S-P-O. It does not serve "which subjects have object `Material`?": the object is the last component, so the matching keys are scattered across the whole tree. The fix is to store the same triples in more than one order.

Hexastore [13] keeps all six permutations; RDF-3X [14] also keeps six but compresses them heavily and adds aggregated indexes. Each permutation is another full copy of the triples, so the right number is a measurement, not a default: three cover every pattern as a prefix, and the other three only add sorted orders that some joins can exploit.

Bound positions in the pattern	Answered as a prefix of
subject	SPO
subject, predicate	SPO
predicate	POS
predicate, object	POS
object	OSP
object, subject	OSP
all three	any (a point lookup)
none	any (a full scan)

Table 5.1: Three permutations (SPO, POS, OSP) make every bound/unbound combination a prefix scan.

Key idea

Extra permutations change the *cost* of answering a pattern, never the *answer*. With only SPO, every pattern is still answerable by scanning and filtering; the other permutations turn scans into prefix ranges.

In OntoDB: keys, permutations, and the catalog

OntoDB's `TripleKey` is exactly fig. 5.1: three big-endian `uint64`s, compared with `memcmp`. `PrefixLower(key, n)` and `PrefixUpper(key, n)` produce the bounds for $n = 0, 1, 2, 3$ bound components. The catalog names three B+ tree roots, `"spo"`, `"pos"`, and `"osp"`, and an insert or delete updates every permutation the catalog has, or none. The plan builds SPO first and then *measures* whether POS and OSP pay for their space on real SPARQL workloads before adding them.

Status

The triple key is the next item to implement (plan 0.2). The indexed store and the permutation measurement follow the B+ tree (plans 3.4 and 3.5).

Further reading

Weiss, Karras, and Bernstein [13]; Neumann and Weikum [14], the sections on the dictionary and on permutations.

Chapter 6

The Log-Structured Alternative

A B+ tree updates pages *in place*: an insert finds the right leaf and changes it. The other great family of storage engines never updates in place. A *log-structured merge tree* (LSM tree), introduced by O'Neil et al. [11], turns every write into a sequential append and pays for it later, in the background. RocksDB, LevelDB, Cassandra, and many modern stores are LSM trees. Knowing why OntoDB does *not* use one teaches as much as knowing why it uses a B+ tree.

6.1 How an LSM tree works

1. **Memtable.** Writes go to an in-memory sorted structure (often a skip list), and to a write-ahead log for durability.
2. **Flush.** When the memtable is full, it is written to disk as an immutable sorted file, an *SSTable*, in one sequential write.
3. **Compaction.** SSTables accumulate. A background process merges them into larger SSTables, dropping overwritten and deleted entries. In *leveled* compaction, level $L + 1$ is about ten times larger than level L.
4. **Reads** check the memtable, then SSTables from newest to oldest, and stop at the first hit.
5. **Deletes** write a *tombstone*, a marker that hides older versions until compaction removes both.

A read that misses must look in many files. *Bloom filters*, one per SSTable, answer "is this key possibly here?" in memory with no false negatives, so most files are skipped without a read.

6.2 Three amplifications

Every storage engine trades three costs against each other.

Write amplification Bytes written to the device per byte of user data. An LSM tree rewrites data at every compaction; a B+ tree rewrites a whole page to change a few bytes.

Read amplification Pages read per lookup. A B+ tree reads one path; an LSM tree may consult several levels.

Space amplification Bytes stored per byte of live data. LSM trees hold obsolete versions until compaction; B+ trees hold half-empty pages.

	B+ tree	LSM tree
Writes	random, in place	sequential, batched
Point reads	one root-to-leaf path	memtable, then levels (Bloom filters help)
Range scans	walk linked leaves	merge iterators across levels
Deletes	remove now, merge pages	tombstone now, remove at compaction
Background work	none required	compaction, always
Best for	read-heavy, range-heavy	write-heavy, ingest-heavy

Table 6.1: The two families, in broad strokes.

Key idea

An LSM tree buys fast writes with deferred work. A B+ tree pays for each write immediately, and gets predictable reads and no background compaction in return.

In OntoDB: why a B+ tree

OntoDB's workload is a SPARQL engine's: a bulk load, then queries that are joins of prefix range scans. Range scans over sorted permutations are a B+ tree's best case, and bulk loading builds full, sequential leaves without any compaction. A delete in OntoDB's tree shrinks the tree in place, inside `BPlusTree::Remove`; there is no tombstone for a reader to skip and no background job that rewrites sorted runs.

The LSM ideas still appear in OntoDB, in smaller roles. Its persistent databases (chapter 13) are log-plus-snapshot: every change is appended to a log, and a checkpoint rewrites the whole state as one sorted image, which is a compaction with a single level. And Bloom filters are a natural fit for "has this concept been modified recently?" checks over a log.

Further reading

O'Neil et al. [11]; Petrov [1], chapter 7; Bloom [12].

Part III

Query Processing

Chapter 7

From SPARQL to a Plan

A query states *what* to find. The engine must decide *how*. This chapter follows a SPARQL query from text to an executable tree of operators, and introduces the iterator model that runs it.

7.1 Parsing and binding

The *parser* turns text into an abstract syntax tree: prefixes, the projected variables, the triple patterns, the filters. Syntax that the engine does not support should fail here, with a line and a column, rather than half work later.

The *binder* then resolves names against the database:

- Prefixed names become full IRIs: `ub:Student` becomes `<http://example.edu/univ#Student>`.
- Every constant becomes a dictionary id. For a query, a constant that is not in the dictionary can match nothing; for an insert, it is added.
- Every variable gets a *slot*: a column number, assigned in order of first appearance.

After binding, a row is simply an array of ids indexed by slot. The same variable in two patterns is the same slot, and that shared slot is the join key.

7.2 Logical and physical plans

A *logical plan* says what to compute in terms of algebra: scan each pattern, join the results, filter, project. A *physical plan* picks an algorithm for each step: which index and range to scan, which join algorithm, in what order. The planner produces a correct physical plan; the optimizer (chapter 8) rewrites it into a fast one that returns the same rows.

For the query

```
SELECT ?name WHERE {
  ?s a ub:Student . ?s ub:name ?name . ?s ub:age ?age .
  FILTER (?age > 21)
}
```

a straightforward plan is left-deep, joining the patterns in the order written:

```
Projection [?name]
  Filter (?age > 21)
    NestedLoopJoin
      NestedLoopJoin
```

```
      TripleScan {?s rdf:type ub:Student}
      TripleScan {?s ub:name ?name}
    TripleScan {?s ub:age ?age}
```

7.3 The iterator (Volcano) model

Graefe's *Volcano* model [15] gives every operator the same three-call interface:

- `Init()`: prepare, or restart from the beginning.
- `Next(&row)`: produce the next row and return true, or return false when exhausted.
- destruction: release everything held (for a scan, its pinned leaf).

The root operator calls `Next` on its child, which calls `Next` on its children, and so on down to the scans. Rows flow up *one at a time*, on demand. Three properties make this model the default for decades:

Composability. Any operator can sit above any other, because they share an interface.

Pipelining. A row can travel from a scan to the client without any intermediate result being materialized. `LIMIT 10` stops the whole tree after ten rows.

Bounded memory. Only operators that must see all their input first (sort, hash-join build, aggregation) hold rows.

Its cost is one virtual call per row per operator. Modern analytical engines reduce that with *vectorized* execution (batches of rows per call) or by compiling the plan into tight loops, but the logical structure is unchanged.

Key idea

Planning builds a tree; execution pulls rows through it. Keeping the two separate is what allows `\explain` to print a plan without running it, and an optimizer to swap one tree for another.

7.4 The operators

Triple scan. Reads the triples that match one pattern. With an index, the bound positions choose a permutation and a prefix range (chapter 5). If a variable occurs twice in one pattern (`?x ex:knows ?x`), the scan also checks that those positions are equal.

Join. Combines rows from two children whose shared slots agree (chapter 8).

Filter. Passes rows whose expression is true. In SPARQL a type error makes the expression false, not the query fail.

Projection. Keeps the selected slots.

Distinct, limit, offset, sort. Distinct applies to *projected* rows (two triples can share a name). Sort compares values, not ids: ids are assigned in load order, which is not value order.

In OntoDB: planning and execution

OntoDB's binder assigns slots in first-appearance order and represents "not yet bound" with a reserved id. The planner always produces the left-deep tree in textual order with nested-loop joins, which is correct but rarely fast; `\explain` prints it. An executor factory maps each plan node to its executor, and the engine pulls rows with `Init/Next`. An empty pattern, with no triple to match, returns one empty row rather than none: the identity input for a join that has nothing to bind yet.

Status

Parsing, binding, planning, triple scans, nested-loop join, filter, and projection run today against the in-memory store. Distinct, limit, offset, and sort are parsed and planned; their executors are next (plans 4.4 and 4.5).

Further reading

Graefe [15]; the W3C SPARQL 1.1 recommendation [18].

Chapter 8

Joins and Query Optimization

A SPARQL query of k triple patterns is a k-way join, and its running time depends far more on *how* it is joined than on anything else. This chapter covers the three join algorithms, how an optimizer estimates the size of intermediate results, and how it searches for a good join order.

8.1 Three join algorithms

Let the outer input have $|R|$ rows and the inner input $|S|$ rows.

Nested-loop join. For each outer row, scan the whole inner input and keep the pairs whose shared slots agree. Cost: $|R| \times |S|$ comparisons, no extra memory. Correct for any join, and the fallback when nothing better applies.

Index nested-loop join. For each outer row, substitute its bindings into the inner pattern and *probe an index* for just the matching triples. If the outer row binds the subject and SPO exists, the inner side becomes a prefix range, often one or a handful of triples. Cost: $|R|$ index probes. This is the common, fast case in SPARQL: one pattern finds the entities, and each further pattern looks up an attribute of each entity.

Hash join. *Build* a hash table on the shared slots of one input (ideally the smaller), then *probe* it with each row of the other. Cost: linear in $|R| + |S|$, plus memory for the build side. It wins when both inputs are large and no index makes the probe selective, and it loses when the build side does not fit in memory, or when an index probe was already a point lookup.

Key idea

Index nested loop is fast when the outer side is small and the inner side is indexed on the join key; hash join is fast when both sides are large; nested loop is always correct. An optimizer's job is to know which case it is in.

8.2 Cardinality estimation

To compare plans, the optimizer needs the *cardinality* of each intermediate result: how many rows each operator produces. It estimates them from *statistics* gathered at load time: the total triple count $|T|$, the number of distinct subjects, predicates, and objects, and per-predicate counts.

For a pattern with predicate p and object o bound, the classic estimate assumes the positions are

independent:

$$|\{\widehat{(s,p,o)}\}| \;=\; \frac{|p|\cdot|o|}{|T|},$$

where $|p|$ and $|o|$ are the counts of triples with that predicate and that object. Independence is convenient and frequently wrong. In RDF, predicates and objects are strongly correlated (`rdf:type` goes with class objects; a `name` predicate never has a class as its object), so independence can be off by orders of magnitude.

The *q-error* measures an estimate $\hat{c}$ against the true count c:

$$q(\hat{c},c) = \max\left(\frac{\hat{c}}{c},\ \frac{c}{\hat{c}}\right) \geq 1.$$

It treats over- and underestimates symmetrically, and errors multiply as they propagate up a plan: a join of two inputs each off by 10 can be off by 100.

8.3 Searching for a join order

A left-deep plan over k patterns has $k!$ orders, and each join can use several algorithms. Selinger's System R optimizer [16] made the search tractable with dynamic programming:

```
for each single pattern P:   best[{P}] = cheapest access path for P
for size = 2 to k:
  for each subset S of patterns with |S| = size:
    for each pattern P in S:
      candidate = join(best[S - {P}], P) with the cheapest algorithm
      if cost(candidate) < cost(best[S]): best[S] = candidate
return best[all patterns]
```

It keeps one best plan per subset rather than per order, which reduces $k!$ to about $k \cdot 2^k$. A simpler *greedy* strategy (start with the most selective pattern, then repeatedly add the cheapest connected pattern) is a good first step and often good enough for the star-shaped joins common in SPARQL.

8.4 Explain analyze

The single most useful optimizer tool is to *run* the plan and print, next to each operator's estimated rows, the rows it actually produced. The node where the two diverge most is where a statistic is missing, usually a correlation that the independence assumption averaged away.

In OntoDB: the optimizer

OntoDB's optimizer takes the planner's tree and returns another tree that must return the same rows. The shell setting `\set join auto|nlj|inlj|hash` either lets it choose or forces one algorithm, and forcing happens in the optimizer, not the executor, so the plan that `\explain` prints is always the plan that runs. Statistics are collected at load: the triple count, distinct counts per position, per-predicate counts, and counts for partially bound patterns.

Estimates are finite, never negative, and binding one more position never increases them. Join ordering starts greedy and left-deep.

Status

Today the planner always joins in textual order with nested loop. Rebinding the inner scan, index nested-loop join, hash join, statistics, cardinality, join ordering, and `\explain analyze` are plans 4.1 to 4.3 and 5.1 to 5.5.

Further reading

Selinger et al. [16]; Graefe [15]; Leis et al. [17] on how good cardinality estimates really are; Neumann and Weikum [14] on RDF selectivity.

Part IV

Transactions

Chapter 9

Transactions and Isolation

A *transaction* is a group of reads and writes that the database treats as one unit. Transactions let an application reason about one operation at a time, even though many run concurrently and any of them may be interrupted by a crash.

9.1 ACID

Atomicity. All of a transaction's writes happen, or none do. A transaction that aborts, or is cut off by a crash, leaves no trace. (Provided by the log: chapters 11 and 12.)

Consistency. A transaction takes the database from one valid state to another, where "valid" is defined by the application's invariants. The engine helps with constraints, but consistency is mostly the application's promise.

Isolation. Concurrent transactions do not see each other's intermediate states; the result is as if they ran in some order. (Provided by concurrency control: chapter 10.)

Durability. Once a commit is acknowledged, its effects survive any later crash. (Provided by the log and `fsync`.)

9.2 Schedules and serializability

A *schedule* is the interleaved order in which the operations of concurrent transactions actually execute. A schedule is *serializable* if its effect equals that of *some* serial order of the same transactions. Serializability is the gold standard of isolation: the application cannot tell that concurrency happened.

Two operations *conflict* if they come from different transactions, touch the same item, and at least one is a write. A schedule is *conflict-serializable* if it can be turned into a serial one by swapping adjacent non-conflicting operations. The test is a graph: draw an edge $T_i \to T_j$ for every pair of conflicting operations where T_i's comes first. The schedule is conflict-serializable exactly when the graph has no cycle.

9.3 Anomalies

Weaker isolation admits specific bad interleavings. Naming them is the clearest way to define the isolation levels [19].

Dirty read. T_2 reads a value T_1 wrote but has not committed. If T_1 aborts, T_2 read data that never existed.

Non-repeatable read. T_1 reads an item, T_2 changes it and commits, and T_1 reads it again and sees a different value.

Phantom. T_1 reads the set of items matching a condition ("every student"), T_2 inserts a new matching item and commits, and T_1 repeats the query and sees an extra row.

Lost update. Both transactions read a counter, both add one, both write: one increment disappears.

9.4 Isolation levels

Level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possible	possible	possible
Read committed	—	possible	possible
Repeatable read	—	—	possible
Serializable	—	—	—

Table 9.1: The SQL standard's levels, defined by the anomalies they forbid.

Berenson et al. [19] showed that defining levels by these phenomena alone is ambiguous, and that real systems' levels (notably snapshot isolation, which forbids all three but admits a "write skew") do not fit the table cleanly. The table remains the right first mental model.

Key idea

Isolation is a dial, not a switch. Lower levels allow more concurrency and specific anomalies; serializable allows none. Each level is defined by what it forbids, and each forbidden anomaly corresponds to a specific lock held for a specific time (chapter 10).

In OntoDB: transactions

Every shell statement is its own transaction unless `\begin` opens an explicit one, ended by `\commit` or `\abort`. Isolation is a setting, `\set isolation ru|rc|rr|ser`, defaulting to read committed. The isolation contract is written as executable schedules: twelve specs, each a scripted interleaving of two sessions with expectations such as "session 2 is blocked", "the read returns Dana", or "session 2 is aborted". They cover dirty reads, non-repeatable reads, lost updates, lock upgrades, two deadlock shapes, write–write conflicts, locks released on abort, and phantoms.
In a persistent database today, each update statement (or `\load`) is one atomic, durable transaction (chapter 13).

Status

Statement-level atomicity and durability run today. Explicit multi-statement transactions and isolation levels are plans 7.2 to 7.6.

Further reading

Gray and Reuter [3]; Berenson et al. [19]; Petrov [1], chapter 5.

Chapter 10

Concurrency Control

Isolation levels say *what* must hold; concurrency control is *how*. This chapter covers locking, the approach OntoDB takes: two-phase locking, multi-granularity locks, deadlock handling, phantoms, and the separate problem of protecting the B+ tree's own structure.

10.1 Two-phase locking

A transaction takes a *shared* (S) lock to read an item and an *exclusive* (X) lock to write it. S is compatible with S; X is compatible with nothing. *Two-phase locking* (2PL) adds one rule: a transaction may not acquire any lock after it has released one. Its life has a growing phase and a shrinking phase.

2PL guarantees conflict-serializability: the order in which transactions reach their *lock point* (the moment they hold all their locks) is an equivalent serial order.

Plain 2PL has a flaw. If T_1 releases an X lock before committing and T_2 reads the value, then T_1 aborts, T_2 has read a value that never existed, and must abort too: a *cascading abort*. *Strict 2PL* holds every X lock until commit or abort, which rules this out.

Key idea

Exclusive locks are held to the end of the transaction at every isolation level, because releasing one early lets others see a write that may still be undone. The isolation levels differ only in how long *shared* locks are held, and whether ranges are locked.

Level	Shared locks
Read uncommitted	none
Read committed	released at the end of each statement
Repeatable read	held until commit
Serializable	held until commit, plus range or predicate locks against phantoms

Table 10.1: Isolation levels as locking rules, given strict 2PL for writes.

10.2 Locking at several granularities

Locking every triple is precise but expensive for a query that reads the whole store; locking the whole store is cheap but serializes everything. Gray et al. [20] let a transaction choose its granularity, using *intention* locks on the coarser levels: before locking a key in S, take IS ("intends to share something below") on the index and on the database.

Held \ Requested	IS	IX	S	SIX	X
IS	yes	yes	yes	yes	—
IX	yes	yes	—	—	—
S	yes	—	yes	—	—
SIX	yes	—	—	—	—
X	—	—	—	—	—

Table 10.2: Multi-granularity lock compatibility. SIX is S on the whole node plus the intention to write below it.

The point of intention locks: a transaction that wants to lock a whole index in S can check one lock on the index, instead of every key below it, because any writer below must already hold IX there.

10.3 Deadlock

T_1 holds a lock T_2 wants, and T_2 holds a lock T_1 wants: both wait forever. Engines either *prevent* deadlock (order locks, or abort on conflicts by timestamp, as in wait–die and wound–wait) or *detect* it: maintain a *waits-for graph* with an edge $T_i \to T_j$ when T_i waits for a lock T_j holds, find a cycle, and abort one transaction in it, the *victim*. Choosing the youngest transaction (the one that has done the least work) is a common and deterministic policy.

10.4 Phantoms

Repeatable read locks the triples a transaction read. A phantom is a triple it did *not* read, because it did not exist yet; no lock covers it. Preventing phantoms needs a lock on what *could* match, not on what does:

Predicate locks lock the query's condition itself. Precise, but testing whether two predicates overlap is expensive in general [21].

Key-range (next-key) locks lock the gap between index keys, so an insert into the scanned range must wait. Practical, and natural on a B+ tree.

10.5 Latches are not locks

A B+ tree split rewrites several pages at once. Even a single-transaction engine with many threads must stop two threads from splitting the same page at the same time. That protection is a *latch*: a short-term reader–writer mutex on a page, held for the duration of one tree operation, never for a transaction. Locks protect *logical contents* for transactional isolation; latches protect *physical structure* for thread safety.

The classic latch protocol for B+ trees is *crabbing*: latch the child, and release the parent if the child is *safe*, meaning it cannot split (for an insert) or merge (for a delete) as a result of this

operation. A thread therefore holds latches only on the part of the path a structure change could reach. An *optimistic* variant takes shared latches down to the leaf, latches only the leaf exclusively, and restarts pessimistically if the leaf turns out to need a split. Lehman and Yao's B-link tree [7] goes further and lets readers run without latching the path at all, by following right-links when a split has moved their key.

10.6 The alternative: multiversion concurrency

Locking makes readers and writers wait for each other. *Multiversion concurrency control* (MVCC) keeps old versions of items, so a reader sees a consistent snapshot as of its start time and never waits for a writer. Writers still conflict with writers. The cost is storing old versions and cleaning them up once no reader can need them. Most modern systems use MVCC with snapshot isolation by default.

In OntoDB: concurrency control

OntoDB uses strict 2PL with five lock modes on three granularities: the database, an index (`"spo"`, `"pos"`, `"osp"`), and a key, with FIFO grant order and in-place upgrades that do not jump ahead of waiters. A blocked shell session prints why it is waiting, for example `session 2 waiting: X lock held by txn 5`. A background thread builds the waits-for graph and aborts the *youngest* transaction in each cycle (the larger id), so deadlock outcomes are deterministic and testable. Serializable adds one phantom mechanism, chosen and documented in the code. B+ tree operations use latch crabbing, and the whole feature runs under ThreadSanitizer. MVCC is a deliberate future comparison, not part of the plan.

Status

Concurrency control is plans 7.1 to 7.6: crabbing, transactions, the lock manager, isolation, deadlock detection, and phantoms. The isolation harness already exists and checks itself.

Further reading

Gray et al. [20]; Eswaran et al. [21]; Lehman and Yao [7]; Bayer and Schkolnick [8]; Petrov [1], chapters 5 and 6.

Part V

Recovery

Chapter 11

Logging and the Write-Ahead Rule

The buffer pool steals and does not force (chapter 3). So at the moment of a crash, the disk can hold changes from transactions that never committed, and can lack changes from transactions that did. The *log* is what lets the engine repair both. This chapter explains what goes in it, and the one rule that makes it trustworthy.

11.1 The failure model

Transaction failure. A transaction aborts: a deadlock victim, a constraint violation, or the user's choice. Its effects must be removed.

System failure. The process or machine stops. Memory is lost; the disk survives. Committed work must reappear; uncommitted work must vanish.

Media failure. The disk is lost. Only a backup or a replica helps; it is outside the scope of the log alone.

The log handles the first two. It is an append-only file of *log records*, each describing one change, written sequentially, which is the access pattern storage handles best.

11.2 Undo and redo

An update record carries enough to apply the change again (*redo*) and to reverse it (*undo*). For a page-oriented engine that might be the bytes before and after; for a triple store it can be logical: "insert triple $(5, 7, 9)$", whose undo is "delete $(5, 7, 9)$".

Every record has a *log sequence number* (LSN), increasing, unique, and often simply the record's byte offset in the log. Records of one transaction are chained by a *prevLSN* field, so the engine can walk a transaction's history backward when it must undo it.

Every page stores a *pageLSN*: the LSN of the last logged change applied to it. The pageLSN is how recovery knows whether a logged change already reached the page on disk: if pageLSN $\geq$ the record's LSN, it did.

11.3 The write-ahead rule

Key idea

Write-ahead logging (WAL). (1) A page may be written to disk only after every log record describing a change to it is durable: flush the log up to the page's pageLSN first. (2) A transaction commits only when its commit record, and therefore all its earlier records, are durable.

Rule 1 makes steal safe: if an uncommitted change reaches the disk, its undo information is already in the log. Rule 2 makes no-force safe: if a committed change has not reached the disk, its redo information is already in the log. Between them, every possible state of the disk after a crash can be repaired.

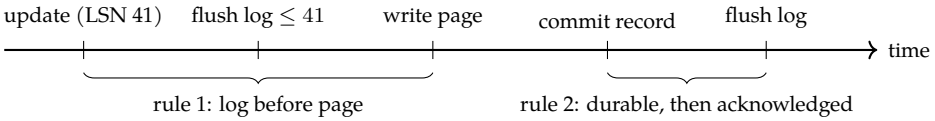


Figure 11.1: The two orderings the write-ahead rule imposes.

11.4 Making commits cheap: group commit

Rule 2 means every commit waits for an `fsync` of the log. If each commit issued its own `fsync`, throughput would be bounded by the device's `fsync` rate. *Group commit* lets commits that arrive while a flush is in progress share the next one:

```
function Commit(txn):
  lsn = Append(commit record for txn)       // into the in-memory log buffer
  wait until persistentLSN >= lsn           // a background flusher wakes us

flusher loop:
  wait until someone asked, or a short interval passes
  batch = swap out the log buffer
  write(batch); fsync()                     // one fsync for many commits
  persistentLSN = last LSN in batch; wake all waiters
```

Under concurrency, the number of `fsync` calls grows much more slowly than the number of commits. A single client still pays the full latency of one flush.

11.5 When a record is damaged

A crash can interrupt the last write to the log, leaving a *torn* record: fewer bytes than its header promises. Bit rot or a misbehaving device can damage a record whose bytes are all present. Robust logs detect both with two fields in every record header: its **length**, so a short record is recognized, and a **checksum** over its bytes, so a damaged one is. PostgreSQL, RocksDB, and

InnoDB all put a CRC on every log record. A recovery procedure then has a policy choice, discussed in chapter 13: stop at the first bad record and keep everything before it, or refuse to start.

In OntoDB: log records and the log manager

Every OntoDB log record has this layout (native byte order):

```
u32 size       whole record, header included
u32 crc32c     over every byte after this field
u8  type       begin | commit | abort | update | clr | checkpoint | term
i64 lsn, i64 txn_id, i64 prev_lsn
then, by type:
  update       i32 page_id, u8 is_insert, u64 subject, u64 predicate, u64 object
  clr          the update fields, then i64 undo_next
  checkpoint   u32 n, then n active transaction ids
  term         u64 term_id, u32 n, then n bytes of canonical spelling
```

The checksum is CRC-32C, the same checksum PostgreSQL, RocksDB, and InnoDB use. A short buffer raises an ordinary storage error (a torn write), while a record whose bytes are present but wrong raises a distinct corruption error, so the two cases can be handled differently. LSNs start at 1; reopening a log continues after its last valid record.
The log manager appends records to an in-memory buffer and a background thread performs group commit; a test runs four threads committing twenty times each and requires fewer `fsync`s than commits. A failed `fsync` is reported, never retried, for the reason in chapter 2.

Status

Log records (plan 8.1) and the log manager with group commit (plan 8.2) run today. Enforcing rule 1 in the buffer pool (plan 8.3) waits for the buffer pool.

Further reading

Mohan et al. [2], sections 1–3; Gray and Reuter [3], chapters 9–10; Petrov [1], chapter 5.

Chapter 12

ARIES

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), published by Mohan, Haderle, Lindsay, Pirahesh, and Schwarz in 1992 [2], is the recovery algorithm behind most disk-based database systems. It supports steal and no force, fine-grained locking, and partial rollbacks, and it recovers correctly even if the system crashes again *during* recovery. Its central idea fits in three words: **repeat history, then undo**.

12.1 The data structures

Log records as in chapter 11: LSN, transaction id, prevLSN, and type-specific fields.

pageLSN on every page: the LSN of the last change applied to it.

Transaction table for every active transaction: its status and *lastLSN*, the LSN of its most recent record.

Dirty page table for every page dirty in the buffer pool: its *recLSN*, the LSN of the first record that made it dirty since it was last read from disk. Any change to the page before recLSN is already on disk.

Compensation log records (CLRs) written when a change is undone. A CLR describes the undo, and carries *undoNextLSN*: the next record of that transaction still to undo.

12.2 Checkpoints

Without checkpoints, recovery would read the log from the beginning of time. A *fuzzy checkpoint* writes the transaction table and the dirty page table into the log without stopping the system or forcing any pages, and records the checkpoint's LSN in a well-known place. Recovery then starts from the last checkpoint: the tables in it tell analysis everything that happened before.

12.3 Pass 1: Analysis

Scan the log forward from the last checkpoint, rebuilding the two tables as of the crash:

- On any record of a transaction not in the table, add it. Update its lastLSN.
- On a commit, mark it committed; on an end record, remove it.
- On an update or CLR for a page not in the dirty page table, add the page with recLSN = this LSN.

At the end, every transaction still in the table without a commit is a *loser*. The *redoLSN* is the smallest recLSN in the dirty page table: no earlier change can be missing from disk.

12.4 Pass 2: Redo

Scan forward from redoLSN, and reapply every update and every CLR, for winners and losers alike, unless the change is provably already on disk:

```
for each update or CLR record r, from redoLSN forward:
  if r.page not in dirty page table:          skip   // page was clean at the crash
  if dirtyPageTable[r.page].recLSN > r.lsn:   skip   // flushed after r
  fetch page
  if page.pageLSN >= r.lsn:                   skip   // r already on the page
  apply r to the page; page.pageLSN = r.lsn
```

Redo **repeats history**: it brings the database to exactly its state at the crash, uncommitted changes included. That sounds wasteful, but it makes undo simple: undo now starts from a precise, known state, and can use the same logical operations that a normal abort uses.

12.5 Pass 3: Undo

Undo the losers, newest change first, across all of them at once:

```
toUndo = { lastLSN of each loser }
while toUndo is not empty:
  r = record with the largest LSN in toUndo; remove it
  if r is a CLR:
    if r.undoNextLSN exists: add r.undoNextLSN to toUndo
    else: write an end record for r.txn
  else if r is an update:
    undo r; write CLR(undoNextLSN = r.prevLSN)
    if r.prevLSN exists: add r.prevLSN to toUndo
    else: write an end record for r.txn
```

Key idea

A CLR is redone but never undone. If the system crashes during undo, the next recovery redoes the CLRs (history repeats), then follows their undoNextLSN pointers past everything already undone. Recovery work is therefore bounded no matter how many times the system crashes during recovery.

12.6 A worked example

Two transactions run. T_1 commits; T_2 does not. The buffer pool happened to write page P2 (stealing T_2's uncommitted change), but never wrote P1.

LSN	Transaction	Record	prevLSN
10	T_1	update P1	—
20	T_2	update P2	—
30	T_1	commit	10
40	T_2	update P1	20

crash. On disk: P1 has pageLSN 0; P2 has pageLSN 20.

Analysis. T_1 committed; T_2 is a loser with lastLSN 40. The dirty page table is {P1: recLSN 10, P2: recLSN 20}, so redoLSN is 10.

Redo. LSN 10 on P1: pageLSN $0 < 10$, so redo; P1's pageLSN becomes 10. LSN 20 on P2: pageLSN $20 \geq 20$, so skip; the stolen write already put it there. LSN 40 on P1: pageLSN $10 < 40$, so redo; pageLSN becomes 40. The database is now exactly as it was at the crash, including T_2's two changes.

Undo. toUndo = {40}. Undo LSN 40 on P1, write CLR 50 with undoNextLSN 20. Undo LSN 20 on P2, write CLR 60 with no undoNextLSN, then an end record for T_2. T_1's change survives; T_2 left no trace.

If the system had crashed right after CLR 50, the next analysis would find T_2 with lastLSN 50; redo would reapply CLR 50; undo would read CLR 50, jump straight to LSN 20, and never undo LSN 40 a second time.

12.7 Structure changes: nested top actions

A B+ tree split moves keys between pages. Suppose T inserts a key, which splits a leaf, and later aborts. Undoing "the split" would be wrong: another transaction may have committed an insert into the new leaf since then. The split is a change to the *structure*, which is valid regardless of who caused it; only the *logical* insert belongs to T.

ARIES logs a split as a *nested top action*: the split's page records, followed by a dummy CLR whose undoNextLSN points to the record just before the split. Redo replays the split like any other change. Undo, following the dummy CLR, skips over it. The logical "insert key k" is logged separately, and its undo is the logical "delete key k", performed wherever that key now lives, which is why undo in ARIES is logical while redo is physical.

In OntoDB: ARIES

OntoDB's page-based recovery follows this chapter: steal and no force, page-oriented redo, logical undo of triples, CLRs that are redone and never undone, and B+ tree splits logged as redo-only nested top actions with the triple insert logged separately. On open, recovery prints one line per pass (`analysis`, `redo`, `undo`). The bar for the feature is a crash harness that kills the process at random points (including named crash points inside splits and between a log flush and a page write) and compares the reopened store with an oracle that replayed only acknowledged commits: `scripts/crashtest -seeds 200` must pass.

Status

The log record types used here, including CLRs and checkpoints, exist and serialize (plan 8.1). Abort by the log, checkpoints, the three passes, and the crash test are plans 8.4 to 8.7, after the buffer pool and B+ tree. The crash harness already detects a lost commit and a surviving uncommitted write on deliberately faulty test stores.

Further reading

Mohan et al. [2], the whole paper; Ramakrishnan and Gehrke [4], chapter 18, for a gentler walkthrough; Petrov [1], chapter 5.

Chapter 13

Durability in Practice

ARIES is the general answer for a page-based engine. A simpler engine can make a simpler choice and still be correct. OntoDB's persistent databases, which run today, are such a design: an in-memory database made durable by a **snapshot** plus a **logical, redo-only write-ahead log**. This chapter explains the design through the ideas of the previous chapters, because every choice in it is one of those ideas in a smaller form.

13.1 Where it sits in the steal/force table

The database lives entirely in memory; the files are its durable copy. Nothing is ever written to the data files except a whole snapshot, and a snapshot is taken only between statements, when every change in memory is committed. In the terms of table 3.2:

- **No steal.** Uncommitted changes never reach the snapshot. So recovery never needs to *undo* anything on disk.
- **No force.** A commit does not rewrite the snapshot; it appends to the log. So recovery must *redo* committed changes made since the last snapshot.

No steal plus no force is the *redo-only* cell: the log needs redo information and nothing else, and recovery is a single forward pass.

Key idea

Choosing a cell of the steal/force table chooses the recovery algorithm. Steal and no force need undo and redo (ARIES). No steal and no force need only redo. The price of no steal is that all uncommitted state must fit in memory, which an in-memory database accepts by definition.

13.2 Logical logging

Page-based ARIES logs *physical* or *physiological* changes: this page, these bytes, or this page, this slot. The memory database has no pages, so it logs *logical* operations: "insert triple (18, 9, 4)". Logical records are small and independent of storage layout, and they are safe here only because there is no partially written page that could make a logical operation ambiguous on replay.

Ids in those records must mean the same terms after a restart, so the log also carries the dictionary: a *term record* for each spelling the first time a transaction uses it.

13.3 One statement, one transaction

1. The statement runs against a *recording store*, a decorator around the real store that remembers every triple it actually changed.
2. The log receives **begin**, a **term** record for each new spelling, one **update** per changed triple, and **commit**.
3. The commit is forced (chapter 11) before the shell prints the result.

If the statement fails part way, the recording store's changes are undone in memory, in reverse order, and nothing it needs reaches the log. A statement that changed nothing logs nothing.

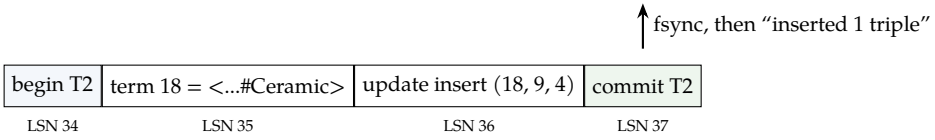


Figure 13.1: The log records of one `INSERT DATA` in a persistent database.

13.4 Recovery

Opening a database directory:

1. Load the snapshot, if there is one: every term in id order, then every triple.
2. Scan the log and stop at the first record that is torn or fails its checksum (chapter 11).
3. Skip records at or below the snapshot's LSN; the snapshot already contains them.
4. Apply *every* term record in order, whatever its transaction did: the dictionary only grows, and a term no committed triple uses is invisible.
5. Apply update records only for transactions that have a commit record, in log order.

Step 5 is the whole of "undo" in a redo-only design: an uncommitted transaction's records are simply never applied.

13.5 Checkpoints as atomic snapshots

A checkpoint writes the full state to `snapshot.tmp`, `fsyncs` it, `renames` it over `snapshot`, `fsyncs` the directory, and then empties the log. Each step is safe to crash after:

Crash after	On reopen
writing `snapshot.tmp`	old snapshot, full log: nothing lost
the rename	new snapshot, old log: records at or below its LSN are skipped
emptying the log	new snapshot, empty log

This is the log-structured idea of chapter 6 with a single level: an append-only log, periodically compacted into one sorted image.

13.6 Damaged logs: recovery modes

When the scan in step 2 stops early, there are two defensible policies, named after RocksDB's WAL recovery modes:

Point in time (the default). Keep every transaction before the bad record, truncate the log there, and report how many bytes were discarded. A torn final record is the normal shape of a crash, and this is the right answer for it; a transaction whose commit is after the damage is treated as uncommitted, exactly as a crash would.

Absolute consistency (`-strict`). Refuse to open, so an operator can inspect the files before anything is truncated.

A damaged snapshot always refuses to open: it has no valid prefix to fall back to.

13.7 Verification

Detecting damage early is the other half of durability. The `\verify` command follows the tradition of PostgreSQL's `pg_amcheck`, MySQL's `CHECK TABLE`, and RocksDB's `VerifyChecksum`: it changes nothing and reports four checks, one line each.

```
snapshot: ok (lsn 37, 18 terms, 15 triples)
wal: ok (0 records, 0 bytes)
replay: matches the live database (15 triples)
invariants: ok (no duplicate triples, every id in the dictionary)
verify: ok
```

The replay check is the strongest: it rebuilds the database from the two files into a fresh store and compares it with the one in memory. If they differ, the files would not reproduce what the user sees.

13.8 Testing a recovery design

A recovery path that is never exercised does not work. OntoDB tests this one with *crash points*: named places in the code, such as just before a commit record is appended and just after a snapshot is renamed, where a test can make the operation stop as if the process died. Scenario tests then reopen the directory and check what survived: a crash before commit leaves no trace, a crash after the snapshot rename does not replay records twice, a torn tail keeps the committed prefix, a flipped byte in the middle of the log drops everything after it, and a tampered log fails verification.

In OntoDB: what runs

All of this chapter runs today: start the shell as `ontodb mydb`. `\load` and every update are durable transactions, `\checkpoint` writes a snapshot, `\log` prints the decoded log, `\verify` checks the files, and `\crash` exits without flushing, to try recovery by hand. Seventeen scenario tests cover the cases above, and they pass under AddressSanitizer, UndefinedBehaviorSanitizer, and ThreadSanitizer.
When the indexed store arrives, the same log records, log manager, and group commit carry over; what changes is that pages can be stolen, so recovery gains undo, pageLSNs, and the three passes of chapter 12.

Further reading

Gray and Reuter [3] on redo-only logging; DeWitt et al. [24] on recovery for main-memory databases; the RocksDB WAL recovery-mode documentation [25].

Chapter A

A Map from Chapters to OntoDB

Each chapter's ideas, the OntoDB plan items that implement them, and where the code lives. Plan items are listed in the repository's `PLAN.md`; run `scripts/check <feature>` to build and test one.

Chapter	Plan items	Where
1 The engine	0.1, 0.3	`tools/shell`, `src/common/database.cpp`
2 Pages	1.1, 3.1	`src/storage/disk`, `src/storage/page`, `src/storage/term`
3 Buffer pool	1.2–1.6	`src/buffer`, `src/storage/page/page_guard.cpp`
4 B+ trees	2.1–2.7, 6.1–6.2	`src/storage/index/b_plus_tree.cpp`
5 Indexing RDF	0.2, 3.2–3.6	`src/storage/index/triple_key.cpp`, `src/dictionary`, `src/catalog`, `src/store`
6 LSM trees	—	(contrast only)
7 Plans	4.4–4.6	`src/parser`, `src/binder`, `src/planner`, `src/execution`
8 Joins	4.1–4.3, 5.1–5.5	`src/execution/executors`, `src/optimizer`, `src/stats`
9 Transactions	7.2, 7.4	`src/concurrency`, `test/isolation/specs`
10 Concurrency	7.1–7.6	`src/concurrency`, B+ tree crabbing
11 Logging	8.1–8.3	`src/recovery/log_record.cpp`, `src/recovery/log_manager.cpp`
12 ARIES	8.4–8.7	`src/recovery`, `test/recovery`, `scripts/crashtest`
13 Durability	9.1–9.7	`src/recovery/durability.cpp`, `src/recovery/snapshot.cpp`, `test/durable`

The OntoDB manual (`https://lamng3.github.io/ontodb-docs/`) is the companion to this book: it documents how to use each feature, while this book explains why it is built the way it is.

Bibliography

[1] A. Petrov. *Database Internals: A Deep Dive into How Distributed Data Systems Work*. O'Reilly, 2019.

[2] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[3] J. Gray and A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[4] R. Ramakrishnan and J. Gehrke. *Database Management Systems*, 3rd edition. McGraw-Hill, 2003.

[5] R. Bayer and E. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[6] D. Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[7] P. L. Lehman and S. B. Yao. Efficient locking for concurrent operations on B-trees. *ACM Transactions on Database Systems*, 6(4):650–670, 1981.

[8] R. Bayer and M. Schkolnick. Concurrency of operations on B-trees. *Acta Informatica*, 9(1):1–21, 1977.

[9] E. J. O'Neil, P. E. O'Neil, and G. Weikum. The LRU-K page replacement algorithm for database disk buffering. In *SIGMOD*, 1993.

[10] W. Effelsberg and T. Härder. Principles of database buffer management. *ACM Transactions on Database Systems*, 9(4):560–595, 1984.

[11] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4):351–385, 1996.

[12] B. H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[13] C. Weiss, P. Karras, and A. Bernstein. Hexastore: Sextuple indexing for semantic web data management. *Proceedings of the VLDB Endowment*, 1(1):1008–1019, 2008.

[14] T. Neumann and G. Weikum. RDF-3X: A RISC-style engine for RDF. *Proceedings of the VLDB Endowment*, 1(1):647–659, 2008.

[15] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–169, 1993.

[16] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *SIGMOD*, 1979.

[17] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.

[18] S. Harris and A. Seaborne, editors. SPARQL 1.1 Query Language. W3C Recommendation, 2013.

[19] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. In *SIGMOD*, 1995.

[20] J. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks and degrees of consistency in a shared data base. In *Modelling in Data Base Management Systems*, 1976.

[21] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger. The notions of consistency and predicate locks in a database system. *Communications of the ACM*, 19(11):624–633, 1976.

[22] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *OSDI*, 2014.

[23] PostgreSQL Global Development Group. Fsync errors ("fsyncgate"). PostgreSQL wiki, 2018. `https://wiki.postgresql.org/wiki/Fsync_Errors`.

[24] D. J. DeWitt, R. H. Katz, F. Olken, L. D. Shapiro, M. R. Stonebraker, and D. Wood. Implementation techniques for main memory database systems. In *SIGMOD*, 1984.

[25] RocksDB. WAL recovery modes. RocksDB wiki. `https://github.com/facebook/rocksdb/wiki/WAL-Recovery-Modes`.